

C++ Programming

Trainer: Nilesh Ghule

C++

→ object oriented language

OOPs concept
~~~~~

↳ practical base

C → system level

C++

# What You'll Learn



## 1. C++ Fundamentals



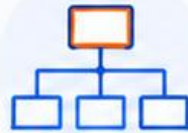
## 2. Object Oriented Programming



## 3. Memory Management



## 4. Operator Overloading



## 5. Inheritance & Polymorphism



## 6. C++ Under the Hood



## 7. Templates



## 8. Exception Handling



## 9. STL & File IO





# Under the Hood

This is where your course becomes different.



1. Compilation and Execution Process



2. Object Memory Layout



3. Virtual Function Internals



4. Virtual Base Class Internals



5. Multiple Inheritance Internals



6. Templates Internals

# Limitations of C Programming

- C is said to be process oriented, structured programming language.
- When program becomes complex, understating and maintaining such programs is very difficult.
- Language don't provide security for data.
- Using functions we can achieve code reusability, but reusability is limited. The programs are not extendible.

# Structure

- Structure is a collection of similar or dissimilar data. It is used to bind logically related data into a single unit.
- This data can be modified by any function to which the structure is passed
- Thus there is no security provided for the data within a structure.
- This concept is modified by C++ to bind data as well as functions.
- By default all members in structure are accessible everywhere in the program by dot(.) or arrow(→) operators.
- But such access can be restricted by applying access specifiers
  - private: Accessible only within the struct
  - public: Accessible within & outside struct

# Comparing struct in C and C++



```
struct time {  
    int hr, min, sec;  
};  
  
void display( struct time *p) {  
    printf("%d:%d:%d", p->hr, p->min, p->sec);  
}
```

```
struct time t;  
display(&t);
```

```
struct time {  
    int hr, min, sec;  
    void display(){  
        printf("%d:%d:%d", hr, min, sec);  
    }  
};
```

```
time t;  
t.display();
```

# What is class?

- Building block that binds together data & code
- Program is divided into different classes
- Class has
  - Variables (data members)
  - Functions (member functions or methods)
- By default class members are private( not accessible outside class scope)
- Classes are stand-alone components & can be distributed in form of libraries
- Class is blue-print of an object



# What is object?

- Object is an instance of class
- Entity that has physical existence, can store data, send and receive message to communicate with other objects
- Object has
  - Data members (***state*** of object)
  - Member function (***behavior*** of object)
  - Unique address(***identity*** of object)
- The values stored in data members of the object called as 'state' of object
- Behavior is how object acts & reacts, when its state is changed & operations are done
- Behavior is decided by the member functions. Objects behaves differently depending upon the state of the object.
- Operations performed are also known as messages

# Comparing struct and class in C++



```
struct time {  
    // default - public  
    int hr, min, sec;  
    void display(){  
        printf("%d:%d:%d", hr, min, sec);  
    }  
};
```

```
time t1;  
t1.display();  
time t2(10, 10, 40);  
t2.display();
```

```
class time {  
    private: // default - private  
        int hr, min, sec;  
    public:  
        void display(){  
            printf("%d:%d:%d", hr, min, sec);  
        }  
};
```

```
time t1;  
t1.display();  
time t2(10, 10, 40);  
t2.display();
```

# Procedure oriented vs Object oriented

- Emphasis on steps or algorithm
- Programs are divided into small code units i.e. functions
- Most functions share global data & can modify it
- Data move from function to function
- Top-down approach
- Emphasis on data of the program
- Programs are divide into small data units i.e. classes
- Data is hidden & not accessible outside class
- Objs communicate with each other
- Bottom- up approach

# Class data members & member functions

- Data members of the class are generally made as private to provide the data security.
- The private members cannot be accessed outside the class.
- So these members are always accessed by the member functions.
- Member functions are generally declared as public members of class.
- Constructor : Initialize Object
- Destructor : De-initialize Object
- Mutators : Modifies state of the object
- Inspectors : Don't Modify state of object
- Facilitator : Provide additional facilities like IO

# Comments in C++

- In C, comments are written as
- `/*This is comment*/`
- In C++, we can use above style. In addition, C++ provides one more way for writing comments.
- `//This is comment`
- The second style is used for single line comments.
- Comments are removed by the preprocessor (before compilation).





Thank You!

Nilesh Ghule

<nilesh@sunbeaminfo.com>